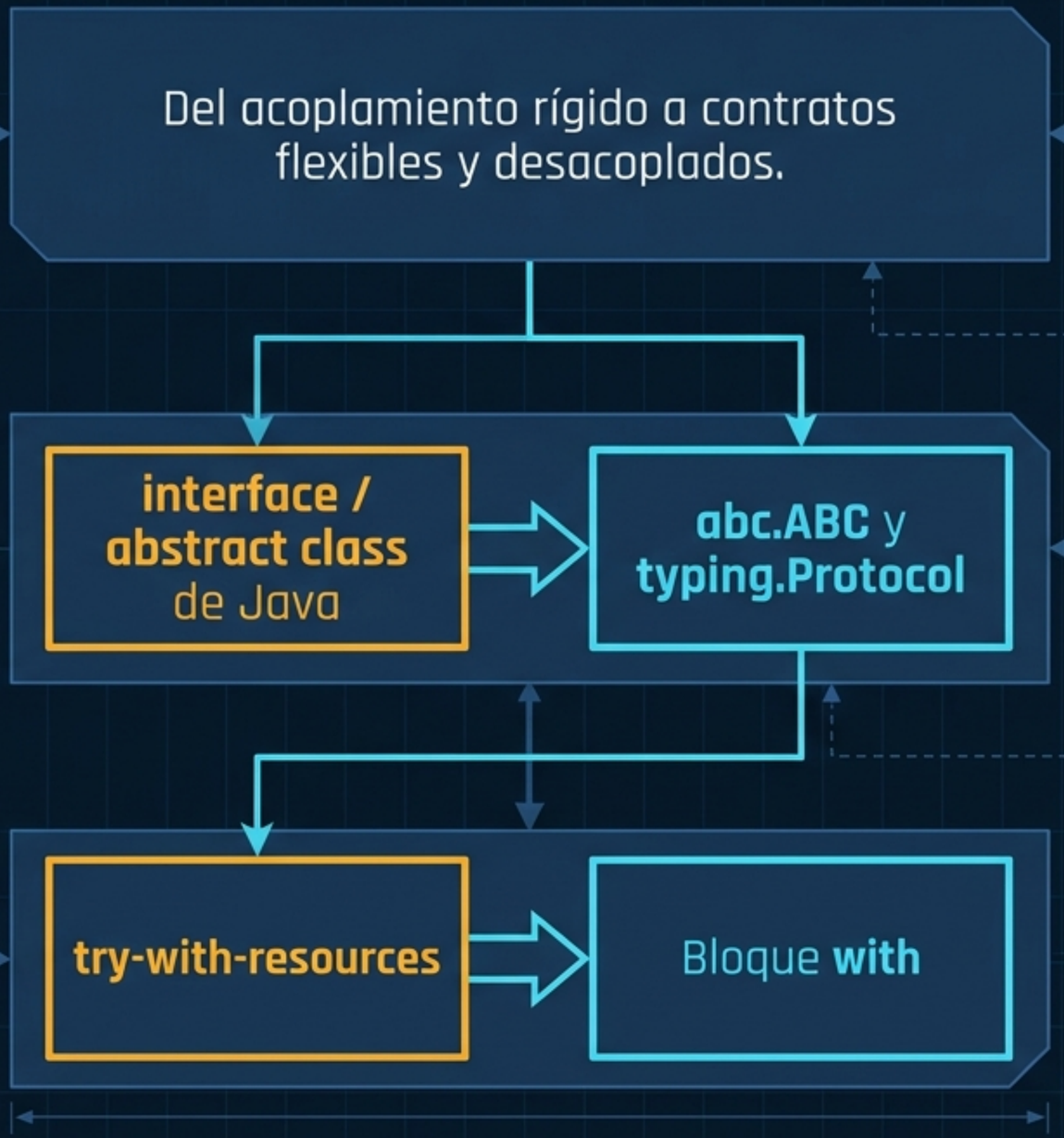
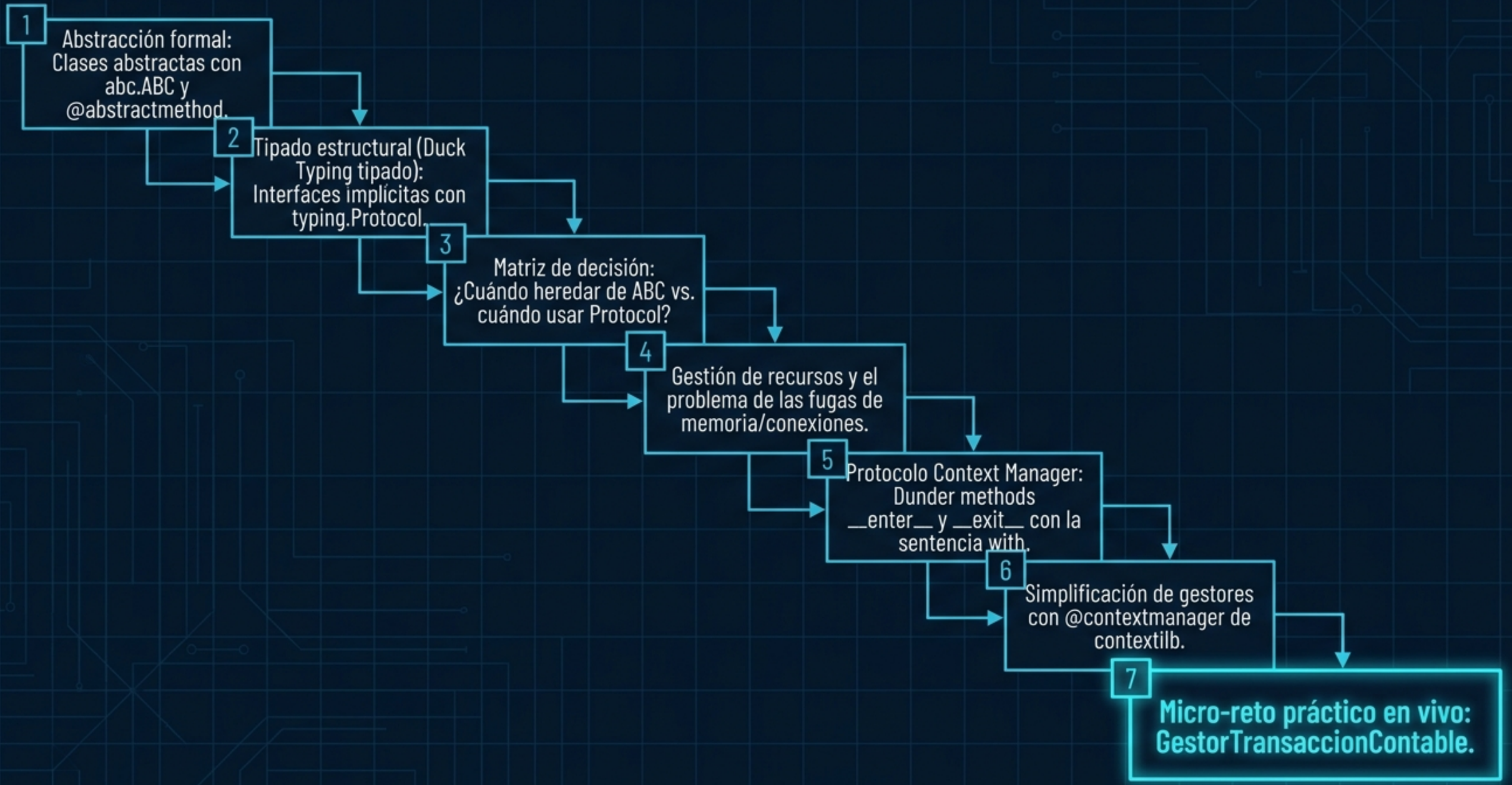
Módulo: Sistemas de Gestión Empresarial (2º DAM)

Contratos, Abstracción y Control de Recursos

Sesión 3: Clases Abstractas, Protocolos y Context Managers



Gemini Notebook



Gemini Notebook

Abstracción en Java vs. Python

Java

- Palabras clave explícitas interface y abstract class.
- Herencia nominal obligatoria (implements RepositorioFacturas).



Python Moderno

- Módulo nativo abc (Abstract Base Classes).
- Decorador @abstractmethod para forzar sobreescritura en subclases.
- Imposibilidad de instanciar la clase base si falta implementar algún método abstracto.

Gemini Notebook

Clases Abstractas Formales con abc.ABC

```python
from abc import ABC, abstractmethod

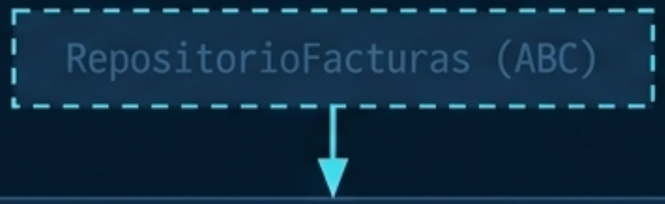
class RepositorioFacturas(ABC):
    @abstractmethod
    def guardar(self, factura_id: str, datos: dict) -> bool:
        """Guarda la factura en el soporte de almacenamiento."""
        pass

    @abstractmethod
    def buscar_por_id(self, factura_id: str) -> dict | None:
        """Recupera la factura o devuelve None si no existe."""
        pass
```

⚠ **Nota**: Intentar instanciar repo = RepositorioFacturas() lanza TypeError: Can't instantiate abstract class

Gemini Notebook

Implementación Concreta de un Contrato




```python
class RepositorioFacturasSQLite(RepositorioFacturas):
    def __init__(self, conexion_db: str) -> None:
        self.conexion_db = conexion_db

    def guardar(self, factura_id: str, datos: dict) -> bool:
        print(f"INSERT INTO facturas VALUES ({factura_id}) en {self.conexion_db}")
        return True

    def buscar_por_id(self, factura_id: str) -> dict | None:
        return {"id": factura_id, "total": 1200.0}
```

Si la subclase olvida implementar `buscar_por_id`, Python impedirá su instanciación en tiempo de ejecución.

Gemini Notebook

Duck Typing y Tipado Estructural

Principio de Duck Typing

Si camina como un pato y suena como un pato, es un pato



Problema tradicional

En Python clásico se asumía que una clase tenía un método sin verificarlo previamente.

Solución moderna (PEP 544)

typing.Protocol (Structural Subtyping).

Nota: Permite definir interfaces sin necesidad de que la clase hija herede explícitamente.

Gemini Notebook

Interfaces Implícitas con typing.Protocol

```python
from typing import Protocol

class Imprimible(Protocol):
    def generar_pdf(self) -> bytes: ...

# ----------------------------------------------------------------------

class FacturaVenta:  # ¡No hereda de Imprimible!
    def generar_pdf(self) -> bytes:
        return b"%PDF-1.4 factura de venta..."

def enviar_por_email(documento: Imprimible) -> None:
    contenido = documento.generar_pdf()
    print(f"Enviando {len(contenido)} bytes...")

enviar_por_email(FacturaVenta())  # Validado correctamente por mypy
```

Gemini Notebook

Matriz Comparativa: abc.ABC vs. typing.Protocol

	abc.ABC (Nominal)	typing.Protocol (Estructural)
Criterio: Relación de Herencia	**Explicita** (`class Hija(Base)`)	**Implícita** (sin herencia requerida)
Criterio: Validación en Runtime	**Sí** (al instanciar bloquea si faltan métodos)	**No** (solo mediante mypy / linters)
Criterio: Reutil-lización de Código	**Permite métodos concretos** con lógica	Solo define **firmas de contrato**
Criterio: Uso Principal en SGE	**Puertos de arquitectura e infraestructura**	**Plugins** y **adaptadores externos**

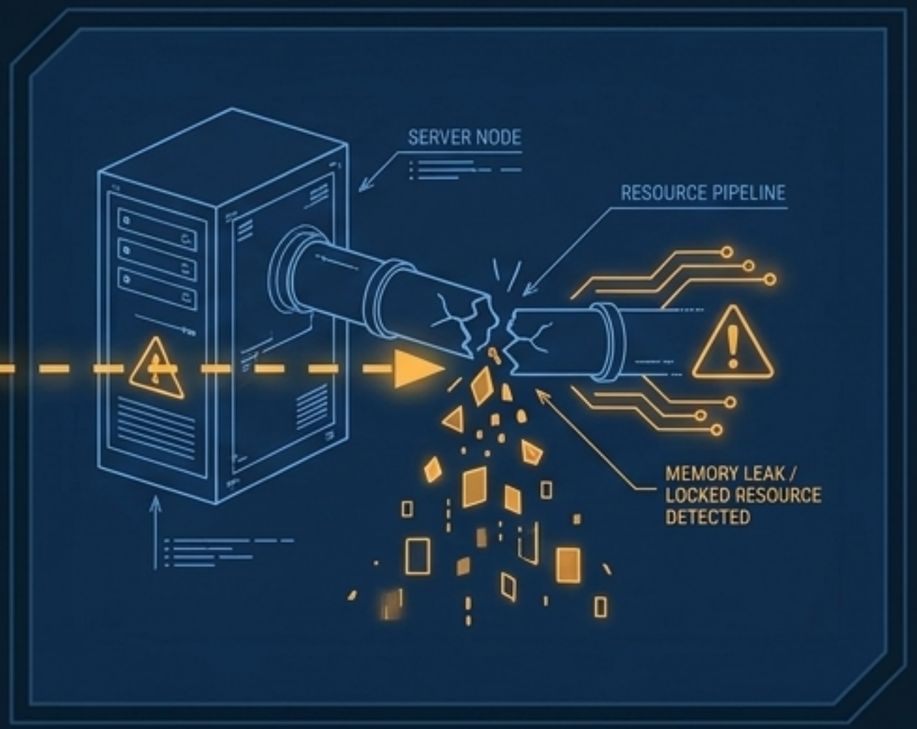
Gemini Notebook

Control de Recursos: Fugas de Memoria y Conexiones

Problema: Abrir ficheros, conexiones a SQLite o sockets de red y olvidar cerrarlos ante una excepción.

Enfoque Imperativo Frágil:

```
f = open("facturas.csv", "w")
# Si ocurre una excepción aquí, f.close() NUNCA se ejecuta
f.write("id,total\n")
f.close()
```



Gemini Notebook

De try-with-resources a la Sentencia with

Java (try-with-resources)

```
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea = br.readLine();
}
```



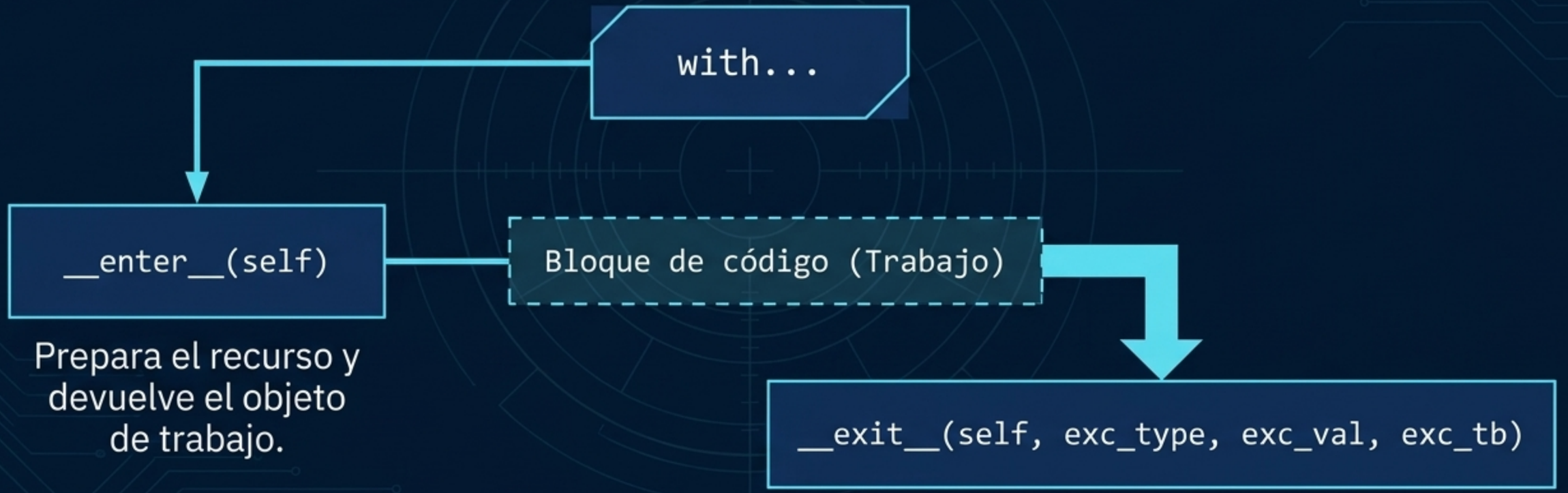
Python (Sentencia with)

```
with open("datos.txt", "r", encoding="utf-8") as archivo:
    linea = archivo.readline()
    # Cierre automático garantizado al salir del bloque, incluso tras un raise
```

Gemini Notebook

El Protocolo Context Manager (__enter__ y __exit__)

Para que un objeto sea compatible con la sentencia `with`, debe implementar dos métodos mágicos:



Prepara el recurso y devuelve el objeto de trabajo.

Captura la salida del bloque y libera el recurso, gestionando excepciones si las hubiera.

Gemini Notebook

Implementación de un Gestor de Transacciones de ERP

```
class TransaccionBaseDatos:
    def __init__(self, conexion: str) -> None:
        self.conexion = conexion

    def __enter__(self) -> "TransaccionBaseDatos":
        print(f"BEGIN TRANSACTION en {self.conexion}")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb) -> bool:
        if exc_type is not None:
            print(f"ROLLBACK debido a: {exc_val}")
            return False  # Propaga la excepción hacia arriba
        print("COMMIT TRANSACTION")
        return True
```

Gemini Notebook

Gestores Ligeros con @contextmanager

```
from contextlib import contextmanager
from typing import Generator

@contextmanager
def auditoria_operacion(modulo: str) -> Generator[None, None, None]:
    print(f" INICIO Operación en módulo: {modulo}")
    try:
        yield  # Cede el control al bloque 'with'
    finally:
        print(f" FIN Operación en módulo: {modulo}")

# Uso limpio en servicios de negocio:
with auditoria_operacion("FACTURACIÓN"):
    print("  --> Calculando libro de IVA...")
```

Gemini Notebook

Caso Práctico Integrador: Almacén y Transacción

```
with TransaccionBaseDatos("sqlite:///erp.db") as tx:
    # Simulación de operaciones de stock
    print("  --> Reservando 5 unidades de SKU-PORT-15...")
    # Si lanzamos un ValueError, la transacción ejecutará ROLLBACK automáticamente
```

Ventaja arquitectónica: Aislamiento completo de la fontanería de base de datos código de negocio.

Gemini Notebook

Micro-reto Práctico en Clase & Solución

Crear el módulo transacciones.py que implemente la clase GestorConexionERP utilizando el protocolo Context Manager:

1. El constructor debe recibir la propiedad db_name: str.
2. __enter__ debe imprimir Conectando a {db_name} y retornar self.
3. __exit__ debe imprimir Cerrando conexion con {db_name}.
4. Implementar un método ejecutar(sql: str) -> None que muestre la consulta por consola.
5. Probar el componente asegurando el cierre automático tras una excepción simulada.

```python
class GestorConexionERP:
    def __init__(self, db_name: str) -> None:
        self.db_name = db_name
    def __enter__(self) -> "GestorConexionERP":
        print(f"Conectando a {self.db_name}")
        return self
    def ejecutar(self, sql: str) -> None:
        print(f"Ejecutando SQL: {sql}")
    def __exit__(self, exc_type, exc_val, exc_tb) -> None:
        print(f"Cerrando conexion con {self.db_name}")

# Prueba de ejecución segura:
with GestorConexionERP("ERP_PROD") as db:
    db.ejecutar("UPDATE stock SET cantidad = 10 WHERE id = 1")
```

Ahora que sabemos definir contratos e interacciones seguras, aplicaremos los Principios SOLID para estructurar servicios de gestión desacoplados.

Gemini Notebook